

AI Agent Systems

Architectures, Patterns & Frameworks

Yao et al. ReAct (2022) | Shinn Reflexion (2023) | Anthropic Agent Docs

Introduction

AI agents are systems where an LLM autonomously takes sequences of actions to accomplish complex goals. This document covers agent architectures, frameworks, and design patterns based on Anthropic's agent documentation and the broader research literature.

1. What Makes an Agent?

An LLM becomes an agent when it can: (1) perceive an environment, (2) take actions that change that environment, and (3) iterate toward a goal over multiple steps. The key shift from chat to agent: the model decides what to do next, not the user.

2. Agent Architecture Patterns

ReAct (Reason + Act): Yao et al. (2022). Interleaves Thought: (reasoning) → Action: (tool call) → Observation: (result). Simple, interpretable. Default pattern in LangChain agents.

Plan-and-Execute: Separate planning agent creates a task list; execution agent carries out each step. Better for long-horizon tasks; more controllable than pure ReAct.

Reflexion: Shinn et al. (2023). Agent reflects on failed attempts and retries with updated strategy. Dramatically improves success rate on coding and reasoning tasks.

Multi-Agent (Hierarchical): Orchestrator agent delegates to specialized subagents (coder, researcher, critic). Enables parallelism and specialization. Used in AutoGen, CrewAI, Claude multi-agent.

Critic/Evaluator: A separate LLM evaluates the primary agent's outputs. Self-critique loop improves quality. Constitutional AI principles applied at inference time.

3. Anthropic's Agent Design Principles

From Anthropic's Build Effective Agents documentation:

- Avoid complexity: use the simplest orchestration pattern that works
- Keep humans in the loop for high-stakes actions

- Use subagents for parallelism, not for artificial complexity
- Prefer reversible actions; pause before irreversible ones
- Inject agent purpose into every subagent prompt to prevent goal drift
- Use structured output parsing, not free-form extraction, for robustness
- Implement circuit breakers: max_steps, max_cost, time limits

4. Claude Agent with Tool Use (Minimal)

```
import anthropic

client = anthropic.Anthropic()

def run_agent(task: str, tools: list, max_steps: int = 10):
    messages = [{"role": "user", "content": task}]

    for step in range(max_steps):
        response = client.messages.create(
            model="claude-opus-4-5",
            max_tokens=4096,
            tools=tools,
            messages=messages,
            system="Complete the task step by step. Use tools when needed."
        )

        messages.append({"role": "assistant", "content": response.content})

        if response.stop_reason == "end_turn":
            # Extract final text response
            return next(b.text for b in response.content if b.type == "text")

        # Handle tool calls
        if response.stop_reason == "tool_use":
            tool_results = []
            for block in response.content:
                if block.type == "tool_use":
                    result = dispatch_tool(block.name, block.input)
                    tool_results.append({
                        "type": "tool_result",
                        "tool_use_id": block.id,
                        "content": str(result)
                    })
            messages.append({"role": "user", "content": tool_results})

    return "Max steps reached without completion"

def dispatch_tool(name, inputs):
    # Your tool router
```

```
tool_map = {"search": search_web, "code": run_code, "read": read_file}
return tool_map[name](**inputs)
```

5. Popular Agent Frameworks

- LangGraph — Graph-based agent workflows; cycles, conditional edges, streaming. Best for complex multi-agent flows.
- AutoGen (Microsoft) — Multi-agent conversation framework. Role-playing agents with GroupChat.
- CrewAI — Crew-based agents with roles, goals, backstories. High-level abstraction.
- Haystack — Open-source NLP framework; pipelines, RAG, agents, evaluation.
- Claude Code — Anthropic's agentic coding tool. Filesystem + bash + editor tools. Production grade.